ERP Backend - Technical Report

Spring Boot 4.0.0 · Java 25 · PostgreSQL 18 · Multi-tenant SaaS

Chapter 1: Project Overview

Module root: erp_backend/erp (Spring Boot application)

Stack: Spring Boot 4.0.0, Java 25, Spring MVC, Spring Data JPA, Spring Security, Lombok

Database: PostgreSQL 18 (Docker container erp-postgres, host port 5433, DB erp_db)

ORM: JPA/Hibernate, ddl-auto=update in dev, open-in-view=false

Architecture: Package-by-feature under com.erp.erp: auth, tenant, user, kyc, billing, common, config

UUID primary keys; DTOs are Java records; entities use Lombok (@Getter/@Setter/@NoArgsConstructor)

Commands: .\mvnw.cmd test, .\mvnw.cmd spring-boot:run, docker compose up -d

Chapter 2: Package & Layering Rules

Layer	Responsibility
Controller	HTTP mapping, validation, request/response only
Service	Business rules, orchestration, transactions
Repository	Database queries only (JpaRepository)
Entity	Persistence only, never exposed through API
Config	Wiring only
Common	Genuinely shared code only

Constructor injection with final fields; no field injection.

Chapter 3: Module: auth - Authentication & Authorization

Entities

- User (users table): id (UUID), fullName, email (unique), passwordHash (BCrypt), role, active, mustRenewPassword, tenant (ManyToOne, nullable - super-admin has no tenant)
- RefreshToken (refresh_tokens table): id, user, tokenHash (SHA-256, unique), expiresAt, revokedAt, createdAt
- PasswordResetCode (password_reset_codes table): id, user (OneToOne), codeHash, expiresAt, attempts, lastSentAt

Role Enum (8 roles)

- SUPER_ADMIN, ADMIN_ENTREPRISE, COMMERCIAL, ACHETEUR, MAGASINIER, COMPTABLE, DIRECTEUR, CLIENT_EXTERNE - each with a code matching the Angular frontend

Repositories

- UserRepository: findByEmail, findFirstByTenantId, existsByEmail
- RefreshTokenRepository: findByTokenHash
- PasswordResetCodeRepository: findByUserId

Services

- AuthService: login, refresh, logout, signupAvailability, signup, me
- PasswordResetService: forgotPassword (6-digit code via email, TTL 15 min, max 5 attempts, 60s cooldown), resetPassword
- RefreshTokenService: create, rotate (revoke + new), revoke (idempotent)
- PasswordResetMailService: sends reset code email via JavaMailSender (Gmail SMTP)
- PlanResolver: derives cheapest plan containing all requested modules (starter/business/enterprise)
- RolePermissions: permission matrix per role (mirrors Angular rbac.ts)

Controllers (/api/v1/auth)

Method	Path	Auth	Description
POST	/login	Public	Returns access + refresh token pair
POST	/refresh	Public	Rotates refresh token, returns new pair
POST	/logout	Public	Revokes refresh token
GET	/signup/availability	Public	Checks subdomain availability
POST	/signup (multipart)	Public	Creates tenant + user + KYC docs; returns Stripe checkout URL
POST	/forgot-password	Public	Sends reset code
POST	/reset-password	Public	Validates code, updates password
GET	/me	Authenticated	Current user + tenant from JWT

Chapter 4: Module: tenant - Multi-tenancy

Entity Tenant (tenants table)

id (UUID), companyName, email, subdomain (unique), country, language, companySize, planId, enabledModules (comma-separated), kycStatus (default APPROVED), decisionNote, decidedAt, billingStatus (default NONE/EXEMPT), billingInterval, stripeCustomerId (unique), stripeSubscriptionId (unique), currentPeriodEnd, cancelAtPeriodEnd, currency (default USD), taxId, address, createdAt

Enum KycStatus: PENDING, APPROVED, REJECTED

Repository TenantRepository

- findBySubdomain, findByStripeCustomerId, findByStripeSubscriptionId, findByKycStatusOrderByCreatedAtDesc, existsBySubdomain, existsByEmail

Chapter 5: Module: kyc - Know Your Customer (signup document review)

Entity SignupDocument (signup_documents table)

- id, tenant (ManyToOne), slot (DocumentSlot), originalFileName, contentType, sizeBytes, storagePath, uploadedAt; unique constraint on (tenant_id, slot)

Enum DocumentSlot: ID_CARD_RECTO, ID_CARD_VERSO, TRADE_REGISTER, TAX_REGISTRATION

Repository SignupDocumentRepository

- findByTenantId, findByIdAndTenant_Id, findByTenantIdAndSlot

Service KycStorageService

- Local filesystem storage: {storageDir}/{tenantId}/{userId}/{documentId}.{ext}
- Allowed types: image/png, image/jpeg, application/pdf; max 2 MB per file
- Validates extension + magic-byte signature; path-traversal protection

Service SignupReviewService (super-admin)

- list(status), download(tenantId, documentId), approve(tenantId) - requires billing ACTIVE/EXEMPT, reject(tenantId, note)

Controller KycAdminController (/api/v1/admin/signups)

Method	Path	Auth	Description
GET		super-admin	List signups by KYC status
GET	/{tenantId}/documents/{documentId}	super-admin	Download a KYC document
POST	/{tenantId}/approve	super-admin	Approve signup (sets KYC APPROVED)
POST	/{tenantId}/reject	super-admin	Reject signup with note

Chapter 6: Module: billing - Stripe Subscriptions

Enums

- BillingStatus: NONE, PENDING, ACTIVE, PAST_DUE, CANCELED, EXEMPT - allowsPlatformAccess() (ACTIVE or EXEMPT), allowsKycApproval(), fromStripe(String)
- BillingInterval: MONTHLY, ANNUAL - fromStripe(String) ("month"/"year")

Entity StripeEvent (stripe_events table)

- id (String), type, processedAt - idempotency for webhook retries

Repository StripeEventRepository: JpaRepository<StripeEvent, String>

Services

- BillingService: checkout (creates Stripe Checkout session, sets PENDING), subscription (status), portal (Billing Portal session), deleteCustomerQuietly
- StripeWebhookService: verifies signature, processes events, idempotent via StripeEventRepository
- StripePriceCatalog: maps plan+interval -> Stripe Price ID from env
- SubscriptionSync: copies Stripe subscription fields onto Tenant

Controllers

Method	Path	Auth	Description
POST	/billing/checkout-session	Authenticated	Create Stripe Checkout session
GET	/billing/subscription	Authenticated	Get billing status
POST	/billing/portal-session	Authenticated	Create Billing Portal session
POST	/webhooks/stripe	Public (signature verified)	Stripe webhook handler

Chapter 7: Module: config & common

SecurityConfig (stateless JWT)

- Public paths (permitAll): login, refresh, logout, signup, signup/availability, forgot-password, reset-password, webhooks/stripe
- /api/v1/auth/me: authenticated
- /api/v1/admin/**: hasAuthority(super-admin)
- /api/v1/billing/**: hasAnyAuthority(kyc:pending, kyc:approved)
- /api/**: hasAuthority(access:full)
- anyRequest: authenticated
- JWT converter: adds role authority, kyc:<status> authority, and access:full when kyc=approved AND billing active/exempt
- 403 handler: returns descriptive message (registration pending approval, registration rejected, payment required, access denied)
- HS256 JWT encoder/decoder, BCryptPasswordEncoder, CORS (localhost:4200)

JwtService

- Generates HS256 tokens with claims: userId, role, permissions, kycStatus, billingStatus, tenantId
- Access token expiration: 15 min; refresh: 30 days

GlobalExceptionHandler (@RestControllerAdvice)

- AuthException -> { message: "..." } with the exception's status
- Validation errors -> 400 with first field error
- Type mismatch -> 400; Missing multipart part -> 400; Max upload size -> 413; Unexpected -> 500 internal error

AuthException: RuntimeException carrying an HTTP status code

ERP Backend - Technical Report

Spring Boot 4.0.0 · Java 25 · PostgreSQL 18 · Multi-tenant SaaS

Chapter 8: API Summary (all /api/v1/)

Auth

```
POST   /auth/login
POST   /auth/refresh
POST   /auth/logout
GET    /auth/signup/availability?subdomain=...
POST   /auth/signup          (multipart: data + 4 KYC files)
POST   /auth/forgot-password
POST   /auth/reset-password
GET    /auth/me
```

Billing (authenticated; tenant derived from JWT)

```
POST   /billing/checkout-session   { interval: "MONTHLY"|"ANNUAL" }
GET    /billing/subscription
POST   /billing/portal-session
```

Webhooks (public)

```
POST   /webhooks/stripe   (Stripe-Signature header required)
```

Admin (super-admin only)

```
GET    /admin/signups?status=PENDING
GET    /admin/signups/{tenantId}/documents/{documentId}
POST   /admin/signups/{tenantId}/approve
POST   /admin/signups/{tenantId}/reject   { note }
```

Chapter 9: Configuration (application.properties)

Property	Default	Description
server.port	8080	
spring.datasource.url	jdbc:postgresql://localhost:5433/erp_db	
spring.datasource.password	${DB_PASSWORD:erpdev1234}	
spring.jpa.hibernate.ddl-auto	update	
app.jwt.secret	dev-only-secret-change-me-32-bytes-min	HS256, override via JWT_SECRET
app.jwt.expiration-ms	900000 (15 min)	
app.jwt.refresh-expiration-ms	2592000000 (30 days)	
app.cors.allowed-origins	http://localhost:4200	Angular frontend
app.kyc.storage-dir	./uploads/images	
app.kyc.max-file-size-kb	2048	
app.password-reset.code-ttl-minutes	15	
app.password-reset.max-attempts	5	
app.password-reset.resend-cooldown-seconds	60	
Stripe keys/URLs/prices	from env (STRIPE_*)	
spring.servlet.multipart.max-file-size	3MB	
spring.servlet.multipart.max-request-size	12MB	

Secrets loaded from optional .env file (gitignored). Never commit real credentials.

Chapter 10: Security Model Summary

- Stateless JWT (HS256), no HTTP sessions
- Passwords hashed with BCrypt; never logged

- Refresh tokens stored as SHA-256 hashes; rotation on use
- Authorization derived from JWT claims: role, kycStatus, billingStatus -> authorities
- Tenant always derived from authenticated user's JWT, never from request body
- Password reset: 6-digit numeric code, email-only, rate-limited, attempt-capped, TTL-enforced
- KYC files: magic-byte validation, path-traversal protection, 2 MB cap, PNG/JPEG/PDF only
- CORS restricted to http://localhost:4200

Chapter 11: Testing

- JUnit 5, @SpringBootTest context load test (ErpApplicationTests)
- Test classes: SignupReviewServiceTest, KycStorageServiceTest, JwtServiceTest, StripePriceCatalogTest, BillingStatusTest, BillingIntervalTest, RefreshTokenServiceTest, PlanResolverTest, PasswordResetServiceTest, AuthServiceTest, RolePermissionsTest
- Run: .\mvnw.cmd test

Chapter 12: Key Design Decisions & Notes

- Multi-tenant by subdomain: each signup creates a Tenant + an ADMIN_ENTREPRISE user; KYC starts PENDING, billing starts NONE
- Payment-gated approval: KYC approval requires billingStatus to be ACTIVE or EXEMPT (Stripe checkout completed)
- Super-admin bypass: SUPER_ADMIN role has empty permissions map and is exempt from KYC/billing checks
- No Flyway yet: schema managed by ddl-auto=update
- Documents stored locally (not S3): {storageDir}/{tenantId}/{userId}/{documentId}.{ext}
- Stripe webhook idempotency: processed event IDs stored in stripe_events table

Chapter 13: Observations / Potential Improvements

- No Flyway/Liquibase for schema migrations (ddl-auto=update is dev-only)
- KYC storage is local filesystem; consider object storage for production
- No refresh-token reuse detection / family revocation on theft
- Password reset codes are per-user (OneToOne), not multiple concurrent codes
- Billing depends on Stripe being configured; endpoints return 503 if secret key missing
- RolePermissions map is hardcoded and must stay in sync with the Angular rbac.ts